

Real-Time CPG Supply-Chain Analytics Warehouse on Google Cloud

A batch + streaming data platform for retail sales, inventory, and shipment analytics

Author	Vivek [Last Name]
Version	1.0
Date	August 2026
Status	Design / In Development
Repository	<a href="https://github.com/<you>/cpg-analytics-warehouse">github.com/<you>/cpg-analytics-warehouse

1. Executive summary

This project delivers an end-to-end data platform on Google Cloud Platform that ingests retail sales, inventory, and shipment data through two paths — scheduled **batch** loads and a real-time **streaming** feed of point-of-sale (POS) transactions — and unifies them into a governed analytics warehouse. Raw data lands in BigQuery, is transformed and tested with dbt into a Kimball-style star schema, and is served to business users through BI dashboards and a custom web dashboard.

The entire stack is provisioned as code with Terraform, deployed through a CI/CD pipeline that runs data-quality tests on every change, and monitored for freshness, cost, and pipeline health. The design follows a **Lambda architecture** (batch layer + speed layer feeding a single serving layer) and a **medallion** data-quality progression (raw → staging → marts, i.e. bronze → silver → gold).

The business framing is deliberately consumer-goods-oriented: the marts answer questions a CPG supply-chain and commercial team actually asks — sell-through by store, inventory position and stockout risk, and shipment-to-sale reconciliation.

2. Problem statement

Consumer-goods organizations generate sales, inventory, and shipment data across many stores, systems, and regions, arriving at different speeds and in inconsistent formats. Back-office systems produce **daily batch files** (finalized, corrected, complete), while store terminals emit **live transaction events** (immediate, but raw and unreconciled). This creates three concrete problems:

1. **Fragmentation.** Sales, inventory, and product-master data live in separate systems with no conformed, queryable single source of truth, so cross-domain questions (e.g. "which fast-selling SKUs are about to stock out?") are slow or impossible to answer.
2. **Latency vs. correctness trade-off.** Leadership wants near-real-time visibility into sell-through, but finance and planning need reconciled, corrected numbers. A single-speed pipeline cannot satisfy both.

3. **Lack of trust and governance.** Without automated data-quality checks, version-controlled transformations, and lineage, analytics outputs are hard to trust and hard to audit — a blocker for data-driven decision-making at scale.

The problem this project solves: build a warehouse and pipeline that ingests multi-speed, multi-source retail data, guarantees data quality through automated testing, models it into business-ready marts, and serves both a *near-real-time* view (from the stream) and a *reconciled* view (from batch) — all reproducibly and cost-consciously.

3. Objectives

The project is considered successful if it achieves the following measurable objectives:

1. **Unified ingestion** — ingest at least three source domains (sales, inventory, product master) via batch and one domain (POS transactions) via streaming, both landing in a single BigQuery warehouse.
 2. **Dimensional model** — deliver a Kimball star schema with at least two fact tables and four conformed dimensions, including a Slowly Changing Dimension (Type 2) on the product master.
 3. **Automated data quality** — enforce not-null, uniqueness, referential-integrity, and accepted-value tests on every model, with the CI pipeline failing on any test failure.
 4. **Infrastructure as code** — provision 100% of cloud resources through Terraform, with no manual console changes required to stand the environment up or tear it down.
 5. **CI/CD** — run `dbt build` plus the full test suite automatically on every pull request, blocking merges on failure.
 6. **Near-real-time serving** — surface streaming POS data in a serving mart with end-to-end latency under a small number of minutes.
 7. **Cost discipline** — run the entire project within GCP free-tier credits by using burst execution, small machine types, and teardown between sessions; document the cost model.
 8. **Observability** — monitor pipeline freshness, throughput, and error rate, with a documented alerting approach.
-

4. Scope

In scope

- Batch ingestion of sales, inventory, and product-master data from Cloud Storage into BigQuery.
- Streaming ingestion of simulated POS transactions via Pub/Sub and Dataflow.
- dbt transformation layer (staging, intermediate, marts) with tests and documentation.
- Star-schema data marts serving both reconciled (batch) and near-real-time (stream) views.
- Terraform-managed infrastructure and a GitHub Actions CI/CD pipeline.
- BI dashboards (Looker Studio / Tableau) and a lightweight React dashboard.
- Monitoring, cost tracking, and project documentation.

Out of scope

- Real production source systems (SAP/BW, live retailer feeds). Sources are realistic synthetic or public sample data; the ingestion layer is written so a real source could be swapped in behind the same interface.
- Machine-learning / demand-forecasting models (noted as a future enhancement).
- Enterprise security hardening, VPC-SC perimeters, and org-level IAM policy (a single-project, least-privilege service-account model is used instead).
- Multi-region deployment.

5. Solution overview

Data enters through two independent paths that converge in the warehouse:

- The **batch path** carries scheduled, finalized data. Files land in a Cloud Storage bucket; Airflow (Cloud Composer) orchestrates loads into BigQuery's raw layer on a schedule.
- The **streaming path** carries live POS events. A generator publishes transaction events to Pub/Sub; a Dataflow (Apache Beam) job processes them — deduplicating, windowing, and handling late/out-of-order events — and writes to the raw layer using the BigQuery Storage Write API for exactly-once delivery.

Both paths land as **separate raw tables** in BigQuery (the convergence point is the warehouse, not dbt). From there, **dbt** owns all transformation: staging models clean and standardize raw data (silver), and mart models build the star schema (gold). Every model carries data-quality tests. The marts feed BI dashboards and a custom web dashboard.

Cross-cutting concerns wrap the whole system: **Terraform** provisions all infrastructure, **GitHub Actions** runs dbt build and tests on every change, **Cloud Monitoring** tracks health and cost, and the codebase is developed with an agentic coding tool (Claude Code) for scoping and refactoring multi-file changes.

The runtime data flow and the build sequence are intentionally different. Data *flows* batch-lane → warehouse → stream-lane merge → marts → serving. The project is *built* batch-lane + dbt spine first (a complete, demoable warehouse), then the streaming lane is added as an enhancement — so there is always a working deliverable.

6. System architecture

Architecture pattern: Lambda (batch layer + speed layer → unified serving layer), with a medallion quality progression inside the warehouse.

BATCH: Sources → Cloud Storage → Airflow (Composer) → BigQuery raw ↙
 ↘ dbt →
 Star-schema marts → BI + React
 STREAM: Sources → Pub/Sub → Dataflow (Beam) → BigQuery raw ↙

Cross-cutting: Terraform (infra) · GitHub Actions (CI/CD) · Cloud Monitoring ·
 Claude Code (development)

Medallion mapping

Layer	Medallion tier	Contents
BigQuery raw tables	Bronze	Untransformed, as-ingested data from both paths
dbt <code>staging/</code> + <code>intermediate/</code>	Silver	Cleaned, typed, deduplicated, conformed
dbt <code>marts/</code> (star schema)	Gold	Business-ready facts and dimensions

7. Technology stack

Layer	Technology	Role / justification
Cloud platform	Google Cloud Platform	Target platform per role; BigQuery-native ecosystem
Batch storage	Cloud Storage	Landing zone / data lake for batch files
Orchestration	Cloud Composer (Airflow)	Scheduled, dependency-aware batch ETL DAGs
Streaming ingest	Pub/Sub	Durable, decoupled event queue for POS events
Stream processing	Dataflow (Apache Beam)	Windowing, dedup, late-data handling, exactly-once writes
Data warehouse	BigQuery	Serverless warehouse; raw + marts; streaming inserts
Transformation	dbt	SQL-based modeling, testing, lineage, documentation
Low-latency serving (optional)	Bigtable	Latest-value lookups for the live dashboard
Infrastructure as code	Terraform	Reproducible, version-controlled provisioning
CI/CD	GitHub Actions (or Cloud Build)	Runs <code>dbt build</code> + tests on every PR
Containerization	Docker	Packages the streaming generator and dbt runtime
Version control	Git / GitHub	Source control, PR-based workflow
BI / visualization	Looker Studio / Tableau	Business dashboards on the marts
Custom dashboard	React + Node/Express	Live web dashboard (differentiator: full-stack build)
Monitoring	Cloud Monitoring / Logging	Freshness, throughput, error rate, cost alerts
Development tooling	Claude Code	Agentic scoping/refactoring across the codebase

Programming languages: Python (generator, Beam pipeline, Airflow DAGs), SQL (dbt models), HCL (Terraform), JavaScript/TypeScript (dashboard).

8. Data model (Gold layer)

The marts use a **Kimball star schema**. Facts hold measurements at a defined grain; dimensions hold descriptive context with surrogate keys.

Fact tables

Table	Grain (one row per...)	Key measures
<code>fact_sales</code>	line item within a POS transaction	quantity, gross amount, discount, net amount
<code>fact_inventory</code>	product × store × day (snapshot)	units on hand, units in transit, reorder point

Dimension tables

Table	Description	Notes
<code>dim_product</code>	SKU, brand, category, pack size	SCD Type 2 — tracks attribute history (e.g. repackaging)
<code>dim_store</code>	store, region, channel, banner	conformed across both facts
<code>dim_date</code>	calendar, fiscal period, week	generated date spine
<code>dim_customer</code>	loyalty member attributes	optional; nullable FK on <code>fact_sales</code>
<code>dim_promotion</code>	promo mechanics and windows	optional; supports promo-lift analysis

Modeling decisions worth defending

- **Surrogate keys** on all dimensions (not natural business keys) to decouple the warehouse from source-system key changes and to support SCD Type 2.
- **SCD Type 2 on `dim_product`** because product attributes change over time and historical sales must reflect the attributes *as they were at time of sale*.
- **Incremental materialization** on `fact_sales` (high-volume) to avoid full rebuilds; snapshots and dimensions rebuilt fully or via dbt snapshots.
- **Conformed dimensions** (`dim_store` , `dim_date`) shared across facts, enabling drill-across analysis.

9. Pipeline design

Batch path. Airflow DAGs run on a schedule: sense/load files from Cloud Storage, load to BigQuery raw tables partitioned by load date, and trigger the downstream dbt run. Idempotent loads and partition-level overwrites make re-runs and backfills safe.

Streaming path. The generator publishes POS events (`transaction_id` , `store_id` , `sku` , `qty` , `price` , `event_ts`) to Pub/Sub. The Dataflow job:

- deduplicates on `transaction_id` (exactly-once semantics),
- applies event-time windowing with watermarks and a bounded allowed-lateness window for out-of-order events,
- routes malformed events to a **dead-letter** topic,
- writes to BigQuery raw via the **Storage Write API**.

A dbt model then unifies the streaming raw table with batch sales into a near-real-time sales mart, so dashboards show reconciled sales through the last batch load plus live sales for the current period.

10. Data quality, testing, and governance

- **dbt tests** on every model: `not_null` and `unique` on keys, `relationships` for referential integrity between facts and dimensions, and `accepted_values` on categorical fields.
- **CI enforcement:** the GitHub Actions pipeline runs `dbt build` (models + tests) on every pull request and blocks merges on any failure.
- **Documentation & lineage:** `dbt docs` generates a browsable model catalog and dependency graph.
- **Code standards:** naming conventions (`stg_`, `int_`, `fct_`, `dim_` prefixes), a `CONTRIBUTING.md`, and a `README` per mart, addressing the requirement to define standards that ensure data quality and integrity.

11. Infrastructure and CI/CD

- **Terraform** provisions the GCP project resources: Cloud Storage buckets, BigQuery datasets (raw/staging/marts), Pub/Sub topics (including dead-letter), service accounts, and least-privilege IAM. `terraform apply` stands the environment up; `terraform destroy` tears it down (also the primary cost control).
- **CI/CD (GitHub Actions):** on PR → lint, `dbt build`, run tests; on merge to main → deploy. Secrets managed via GitHub/Cloud secret storage, never committed.
- **Docker** packages the streaming generator and Beam pipeline for consistent local and Cloud Run execution.

12. Implementation roadmap

Phase	Deliverable	Maps to (JD)
0	Terraform foundation (project, buckets, datasets, IAM)	Terraform, cloud environments
1	Batch ingestion: GCS → Airflow → BigQuery raw	ETL pipelines, Airflow
2	dbt star schema (staging → marts)	Data modeling, dbt, warehousing, marts
3	Data-quality tests, docs, code standards	Data quality & integrity, standards
4	Streaming leg: Pub/Sub → Dataflow → raw → unified mart	Pub/Sub, streaming (preferred)
5	CI/CD, BI + React dashboards, monitoring, packaging	CI/CD, Tableau, observability

Build order deliberately front-loads the batch + dbt core (the required-qualifications cluster); the streaming leg is added once a complete warehouse is demoable.

13. Non-functional considerations

- **Cost.** Dataflow is the primary cost driver and runs continuously only when demoing the live feed; otherwise executed in bursts on the smallest machine type. `terraform destroy` between sessions. Target: within GCP free credits. A cost breakdown is maintained in the README.
 - **Scalability.** BigQuery and Dataflow scale horizontally; partitioning and clustering on high-volume facts keep query cost bounded. The design scales from the demo dataset to production volumes without architectural change.
 - **Reliability.** Idempotent batch loads, dead-letter handling and exactly-once semantics on the stream, and CI gating on data-quality failures.
 - **Observability.** Cloud Monitoring dashboards for pipeline lag, event throughput, error rate, and reconciliation drift; alerting on freshness and cost thresholds.
-

14. Assumptions and constraints

- Source data is realistic synthetic / public sample data standing in for proprietary retailer feeds.
 - Single GCP project, single region, to stay within free-tier limits.
 - Streaming volume is scaled to demo levels; the design (not the running scale) demonstrates production readiness.
 - No proprietary SAP connectors are used; the batch loader is written so an SAP extract could be dropped into the same landing pattern.
-

15. Risks and mitigations

Risk	Impact	Mitigation
Dataflow cost overrun	Credit burn	Burst execution, small workers, teardown, budget alerts
Scope creep (both paths half-built)	No demoable deliverable	Build batch + dbt fully first; stream is additive
Data-quality regressions	Untrusted outputs	dbt tests gated in CI
Streaming late/duplicate events	Incorrect near-real-time numbers	Watermarks, allowed lateness, <code>transaction_id</code> dedup, dead-letter queue
Manual environment drift	Non-reproducible setup	100% Terraform, no console changes

16. Expected outcomes and deliverables

- A reproducible GCP data platform deployable via `terraform apply`.

- A tested dbt project producing a documented star schema.
 - A CI/CD pipeline enforcing data quality on every change.
 - BI dashboards and a live web dashboard on the marts.
 - A repository with this design doc, an architecture diagram, a dashboard demo (GIF/video), a cost breakdown, and a "design decisions & trade-offs" section.
-

17. Interview talking points (project defense)

The project is engineered so the following can be stated with evidence:

- "The convergence point is the raw layer, not dbt — dbt is a transformation tool operating *inside* the warehouse, reading from raw, not from Pub/Sub or GCS."
 - "I guarantee exactly-once into BigQuery with the Storage Write API and `transaction_id` dedup; out-of-order events are handled with watermarks and bounded allowed lateness; malformed events go to a dead-letter queue."
 - "The product dimension is SCD Type 2 so historical sales reflect product attributes as they were at time of sale."
 - "The whole stack is Terraform-managed and every change runs `dbt build` and tests in CI before merge."
 - "I follow a medallion progression, and the Gold layer is modeled as a Kimball star schema — the two frameworks stack rather than compete."
-

18. Future enhancements

- Demand forecasting / stockout-risk ML on the marts.
 - Data-observability tooling (freshness/volume anomaly detection) beyond dbt tests.
 - Reverse-ETL of curated marts back into operational tools.
 - Swap synthetic sources for a real retailer or SAP-style extract behind the existing ingestion interface.
-

19. References / appendix

- Architecture diagram: `docs/architecture.excalidraw` / `docs/architecture.png`
- dbt project: `/transform`
- Terraform: `/infra`
- Streaming pipeline: `/streaming`
- Airflow DAGs: `/orchestration`

Note: "medallion / bronze-silver-gold" is Databricks terminology; on GCP/dbt this project uses the equivalent "raw / staging / marts." Both labels describe the same quality progression.